

RapidAI Research

安妮整理

LLM Harness Engineering — 从理论到实践

核心定义： Agent = Model + Harness。Harness（线束/甲冑）是模型之外的一切——代码、配置、执行逻辑、工具、记忆、沙箱、编排策略。模型提供智能，Harness 让智能有用（The model contains the intelligence and the harness makes that intelligence useful）。

— LangChain, "The Anatomy of an Agent Harness" (2026)

目录

第一篇：理论基础

- 1. 什么是 Harness Engineering?
 - 1.1 从一个隐喻开始
 - 1.2 三层工程体系
 - 1.3 为什么 Harness 如此重要?
- 2. Harness 的完整解剖
 - 2.1 核心组件总览
 - 2.2 从模型需求反推 Harness 设计
- 3. 历史演进：从简单循环到完整 Harness
 - 3.1 第一代：Chatbot Loop (2023)
 - 3.2 第二代：ReAct Loop (2023-2024)
 - 3.3 第三代：状态化 Agent (2024-2025)
 - 3.4 第四代：完整 Harness (2025-2026)

- 3.5 第五代：自然语言 Harness (2026+)

第二篇：核心组件深度剖析

- 4. 规划循环 (Planning Loop)
 - 4.1 ReAct — Agent 的心脏
 - 4.2 编排器-工作者模式 (Orchestrator-Worker)
 - 4.3 Ralph Loop — 跨上下文窗口延续
- 5. 工具设计
 - 5.1 工具分层
 - 5.2 为什么 Bash 是最重要的工具?
 - 5.3 工具描述即编程
- 6. 记忆系统 (Memory in Harness)
 - 6.1 文件系统是最基础的 Harness 记忆原语
 - 6.2 AGENTS.md — 持续学习的载体

项目结构

常用命令

已知问题 (Agent 不要犯这些错)

架构约束

- 6.3 进度文件 — 跨会话状态传递

Session 1 (2026-03-25 10:00)

Session 2 (2026-03-25 14:00)

Session 3 (2026-03-26 09:00)

- 7. 沙箱环境 (Sandbox)
 - 7.1 为什么需要沙箱?
 - 7.2 沙箱的配置层次

第三篇：高级机制与实战方法论

- 8. 上下文腐烂 (Context Rot) 与对抗策略
 - 8.1 什么是上下文腐烂?
 - 8.2 四大对抗策略
- 9. 中间件/钩子系统 (Middleware & Hooks)
 - 9.1 什么是 Harness 中间件?
 - 9.2 实战中间件清单
 - 9.3 LoopDetection 中间件详解
 - 9.4 PreCompletionChecklist 中间件详解
- 10. NLAH — 自然语言 Agent Harness (2026 前沿)
 - 10.1 问题：传统 Harness 不可移植
 - 10.2 NLAH 的五个核心组件
 - 10.3 智能运行时 (IHR)
 - 10.4 NLAH 的实验结果

第四篇：工业实践与案例研究

- 11. Anthropic 的长期运行 Agent 模式

- 11.1 核心问题
- 11.2 双 Agent 解决方案
- 11.3 功能清单的设计智慧
- 12. LangChain 的 Harness 优化实战
- 12.1 从 Top 30 到 Top 5
- 12.2 优化方法论：追踪分析循环
- 12.3 具体改进措施
- 13. OpenAI 的百万行代码 Harness
- 13.1 核心方法
- 13.2 迭代式 Harness 改进
- 14. 核心设计原则汇总

第五篇：研究前沿与开放问题

- 15. Harness 的进化方向
- 15.1 模型与 Harness 的协同进化
- 15.2 开放研究问题
- 15.3 未来展望
- 16. 实践检查清单
- 16.1 构建你的第一个 Harness
- 16.2 Harness 评估维度
- 17. 关键资源索引

第一篇：理论基础

1. 什么是 Harness Engineering?

1.1 从一个隐喻开始

马 (Horse) = 有力量，但不能为你耕田

+ 马具 (Harness) = 缰绳、鞍具、挽具 → 将力量导向有用的工作

= 可用的畜力

大模型 (LLM) = 有智能，但不能为你干活

+ 线束 (Harness) = 工具、记忆、编排、沙箱、护栏 → 将智能导向有用的任务

= 可用的 Agent

Mitchell Hashimoto (HashiCorp 联合创始人) 在 2026 年 2 月的博客中首次系统化了这个概念：

"每次 agent 犯错，你花时间工程化一个解决方案，让 agent 永远不再犯那个特定的错误。这就是 Harness Engineering。"

1.2 三层工程体系

三层工程体系（由窄到宽）

Prompt Engineering（提示词工程）

- └ 范围：写好指令文本
- └ 粒度：单次调用优化
- └ 类比：告诉员工做什么

Context Engineering（上下文工程）

- └ 范围：管理模型看到什么、何时看到
- └ 粒度：跨调用的信息管理
- └ 类比：给员工准备好的工作台和参考资料

Harness Engineering（线束工程）

- └ 范围：模型之外的完整基础设施
- └ 粒度：系统级的设计、编排、运维
- └ 类比：设计整个工作环境、流程、工具链和安全制度

每一层包含前一层：PE ⊂ CE ⊂ HE

"Harness Engineering is the discipline of building the infrastructure around the model so it can do useful work reliably."

— Decoding AI, "Agentic Harness Engineering" (2026)

1.3 为什么 Harness 如此重要？

原因 1：裸模型不是 Agent

裸模型能做的：

输入文本 → 输出文本
(仅此而已)

裸模型不能做的：

- X 维护持久状态
- X 执行代码
- X 访问实时信息
- X 操作文件系统
- X 发送网络请求
- X 自我验证结果
- X 跨会话协作

→ 这些都是 Harness 级别的能力

原因 2：Harness 的差异远大于模型的差异

LangChain 在 TerminalBench 2.0 上的实验：

- 同一个模型 (GPT-5.2-Codex) ，只改 Harness → 从 Top 30 飙升到 Top 5
- 同一个模型 (Opus 4.6) ，不同 Harness → 性能差距巨大
- OpenAI Codex 团队：5 个月打造 Harness → 维护 100 万行代码的产品

"Most engineering teams obsess over which model to use... But the model is only half the equation."

— Decoding AI

原因 3：生产级 Agent 需要系统工程

Demo 级 Agent:

一个 prompt + 几个工具 = 能跑的 demo
(但无法投入生产)

生产级 Agent:

- + 持久化记忆和状态
- + 沙箱化执行环境
- + 错误恢复和重试
- + 多界面接入 (CLI/Web/IDE/IM)
- + 审计日志和可观测性
- + 安全护栏和权限控制
- + 跨会话延续能力
- = 可靠的生产系统

2. Harness 的完整解剖

2.1 核心组件总览



2.2 从模型需求反推 Harness 设计

这是 LangChain 提出的核心方法论：从「模型不能做什么」出发，推导 Harness 需要什么。

模型的局限性	需要的 Agent 行为	Harness 设计
无持久状态	跨会话保持工作进度	文件系统 + 进度文件 + Git
无法执行代码	自主解决问题	Bash 工具 + 代码执行
无法验证自身	检查工作是否正确	测试框架 + 浏览器截图
知识有截止日期	获取实时信息	Web 搜索 + MCP 工具
上下文窗口有限	长时间工作不退化	压缩 + 工具输出截断 + 技能渐进加载
无法感知环境	了解工作目录/工具	环境发现中间件
无法估计时间	在时限内完成	时间预算告警
倾向过早停止	持续工作直到完成	Ralph Loop + 验证清单

3. 历史演进：从简单循环到完整 Harness

3.1 第一代：Chatbot Loop (2023)

```
# 最简单的 "harness" - 就是一个 while 循环
messages = []
while True:
    user_input = input("You: ")
    messages.append({"role": "user", "content": user_input})
    response = llm.chat(messages)
    messages.append({"role": "assistant", "content": response})
    print(f"AI: {response}")
```

局限：只能对话，不能行动。模型只是一个聊天器。

3.2 第二代：ReAct Loop (2023-2024)

ReAct 循环:

观察 (Observe) → 思考 (Think) → 行动 (Act) → 观察 → ...

实现:

```
while not done:
    response = model.chat(context)
    if response.has_tool_call:
        result = execute_tool(response.tool_call)
        context.append(tool_result)
    else:
        return response.text
```

突破: 模型可以调用工具了。但状态仅在内存中，会话结束即丢失。

3.3 第三代：状态化 Agent (2024-2025)

增加的能力:

- + 文件系统读写
- + Bash 代码执行
- + 沙箱环境
- + 上下文压缩
- + 记忆持久化

代表: Claude Code (早期版本)、Cursor、Windsurf。

3.4 第四代：完整 Harness (2025-2026)

增加的能力:

- + 多界面接入 (CLI/Web/IDE/IM/Desktop)
- + Ralph Loop (跨上下文窗口延续)
- + 子 Agent 调度
- + 确定性中间件 (钩子)
- + 自我验证循环
- + 进度追踪和状态恢复
- + 可观测性 (Tracing)

代表: Claude Code (完整版)、Codex (OpenAI)、OpenCode (开源)、OpenClaw (开源)。

3.5 第五代：自然语言 Harness (2026+)

增加的能力：

+ Harness 控制逻辑用自然语言表达（而非代码）

+ 可移植、可比较、可研究

+ 智能运行时解释执行

代表：NLAH（Natural-Language Agent Harnesses，清华，2026）。

第二篇：核心组件深度剖析

4. 规划循环（Planning Loop）

4.1 ReAct — Agent 的心脏

ReAct（Reasoning + Acting）是几乎所有现代 Agent 的基础循环模式。

ReAct 循环

观察

推理

行动

观察

...

Observe

Think

Act

Observe

停止条件满足时

输出最终结果

具体例子 — 修复一个失败的测试：

观察：测试输出显示 ImportError

推理：导入路径可能错误，需要检查文件结构

行动：执行 bash 命令 find . -name "module.py"

观察：发现文件在 src/ 而不是 lib/ 目录下

推理：需要修改导入语句

行动：编辑文件，将 import lib.module 改为 import src.module

观察：重新运行测试，测试通过

推理：任务完成

输出：测试已修复

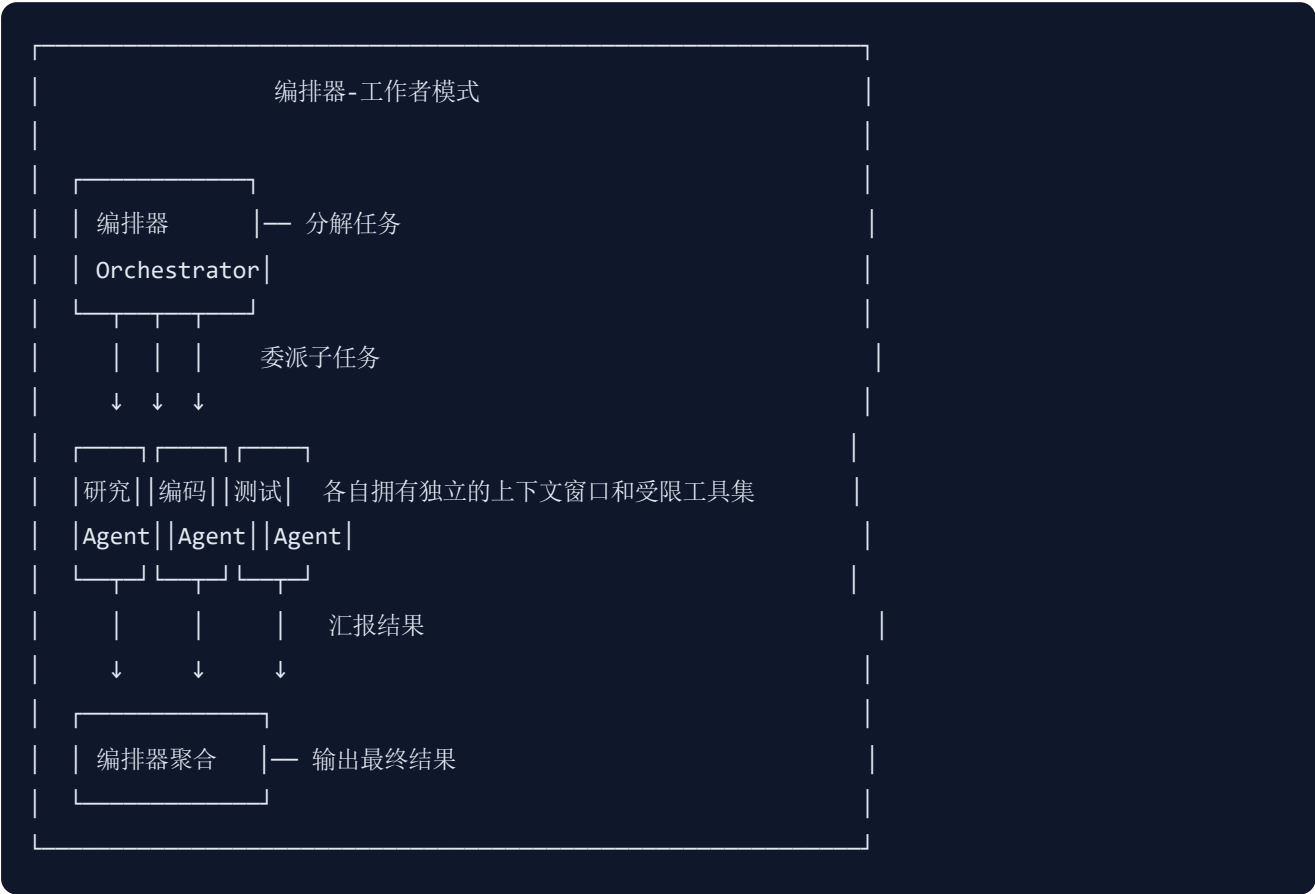
Harness 的角色：模型负责推理和选择行动，Harness 负责：

1. 管理上下文（维护对话历史）

- 2. 执行工具调用 (bash、文件操作等)
- 3. 判断停止条件
- 4. 处理错误和超时

4.2 编排器-工作者模式 (Orchestrator-Worker)

当任务超出单个 Agent 能力时：



关键设计决策：

- 每个工作 Agent 有**独立的上下文窗口**（避免上下文污染）
- 每个工作 Agent 有**受限的工具集**（最小权限原则）
- 文件系统作为**共享协作表面**（Agent 之间通过文件传递中间结果）

4.3 Ralph Loop — 跨上下文窗口延续

这是 Anthropic 发明的关键 Harness 模式，解决了"长任务跨会话"问题：

Ralph Loop

上下文窗口 1（即将满）：

[原始 prompt] → [工作进展] → [工具调用结果] → ... → 上下文快满

↓

Harness 拦截 agent 的退出行为

↓

将状态持久化到文件系统（进度文件 + Git 提交）

↓

上下文窗口 2（全新的）：

[重新注入原始 prompt] → [读取进度文件] → [读取 git log]

→ agent 继续工作 → ... → 上下文又快满了

↓

再次拦截 → 再次持久化 → 再次重开窗口

...（循环直到任务完成）

关键点：

- 每次 Ralph Loop 迭代 = 全新的上下文窗口（不累积历史噪音）
- 通过文件系统/Git 恢复状态（而非尝试传递压缩后的上下文）
- Harness 强制 agent 继续工作（而非让它"宣布完成"）

5. 工具设计

5.1 工具分层

工具分层体系

第一层：通用执行工具

- └ Bash: 万能工具，模型可以写代码并执行
- └ 价值：模型可以自主设计工具，而非被限制在预定义工具中

第二层：专用文件系统工具

- └ read_file: 读文件（强制绝对路径 + 行数限制）
- └ write_file: 写文件（目录自动创建）
- └ edit_file: 精确编辑（验证替换字符串唯一性）
- └ search_files: 搜索内容（正则表达式支持）
- └ list_directory: 列目录
- 比 bash 操作文件更快、更安全、更可预测

第三层：验证与观测工具

- └ 测试运行器（pytest, go test 等）
- └ 代码检查器（linter）
- └ 浏览器自动化（Puppeteer MCP）
- └ 截图工具（视觉验证）
- └ 日志查看器

第四层：状态管理工具

- └ 任务追踪（task list）
- └ 进度记录（progress file）
- └ 计划文件（plan.md）

5.2 为什么 Bash 是最重要的工具？

"Harnesses ship with a bash tool so models can solve problems autonomously by writing & executing code."

— LangChain

```
# 没有 bash: 模型只能调用预定义工具
tools = ["search", "read_file", "write_file"]
# → 遇到未预见的任务时束手无策

# 有了 bash: 模型可以自己写工具
# 模型生成了以下 bash 命令:
"""
python3 -c "
import json, sys
data = json.load(open('config.json'))
# 自己写一个 JSON 处理脚本
for item in data['items']:
    if item['status'] == 'active':
        print(item['id'])
"
"""
# → 模型不再受限于固定工具集
```

安全考量: Bash 是双刃剑。Harness 需要:

- 在沙箱中执行 (隔离环境)
- 命令白名单/黑名单
- 网络隔离 (可选)
- 资源使用限制 (CPU/内存/磁盘)

5.3 工具描述即编程

工具描述 (tool description) 是 Harness 最重要的"代码"之一:

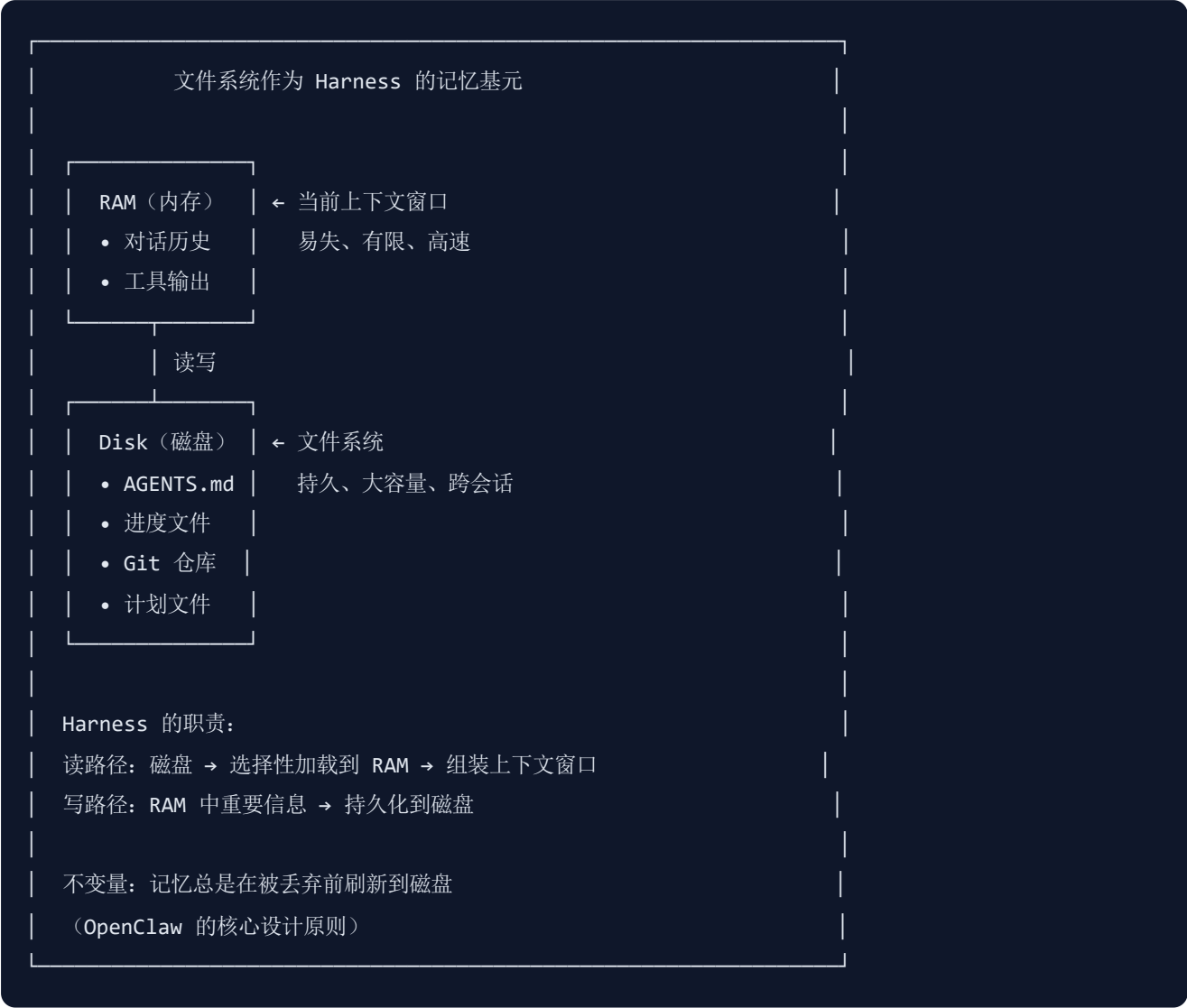
```
{
  "name": "edit_file",
  "description": "对文件进行精确的字符串替换编辑。old_str 必须在文件中唯一匹配。如果匹配到多个位置，",
  "parameters": {
    "type": "object",
    "properties": {
      "file_path": {
        "type": "string",
        "description": "要编辑的文件的绝对路径"
      },
      "old_str": {
        "type": "string",
        "description": "要被替换的原始字符串（必须在文件中唯一）"
      },
      "new_str": {
        "type": "string",
        "description": "替换后的新字符串"
      }
    },
    "required": ["file_path", "old_str", "new_str"]
  }
}
```

设计原则：

- 描述中包含**约束条件**（如"必须唯一匹配"）
- 参数描述包含**使用场景**（而非只是类型信息）
- 错误条件要**提前告知**（让模型知道什么情况下会失败）

6. 记忆系统 (Memory in Harness)

6.1 文件系统是最基础的 Harness 记忆原语



6.2 AGENTS.md — 持续学习的载体

```
# AGENTS.md 示例（来自 Ghostty 项目）

## 项目结构
- src/ 包含核心终端模拟器代码
- src/config/ 包含配置解析，使用 zig 的已知模式

## 常用命令
- 构建项目: zig build
- 运行测试: zig build test
- 运行特定测试: zig build test -Dfilter="test_name"

## 已知问题（Agent 不要犯这些错）
- 不要使用 std.process.exit()，使用正确的错误传播
- 配置文件解析使用自定义 parser，不要引入外部依赖
- macOS 特定代码在 src/macos/ 下，不要放在 src/ 根目录

## 架构约束
- 所有公共 API 必须有文档注释
- 错误处理使用 zig 的 error union 类型
```

Harness 的记忆循环：

```
Agent 工作中发现新模式/错误/最佳实践
→ 更新 AGENTS.md
→ 下次会话开始时，Harness 自动加载 AGENTS.md 到上下文
→ Agent 基于更新后的知识工作
= 持续学习（Continual Learning）通过文件系统实现
```

6.3 进度文件 — 跨会话状态传递

Anthropic 的长期运行 Agent 模式：

claude-progress.txt 示例

Session 1 (2026-03-25 10:00)

- 初始化项目结构 (Next.js + TypeScript)
- 实现基础路由和布局组件
- 设置开发服务器脚本 `init.sh`
- 创建 `feature_list.json` (共 200+ 功能点, 全部标记为 `failing`)
- Git commit: "init: project scaffolding and feature list"

Session 2 (2026-03-25 14:00)

- 实现"新建对话"功能 (`feature_list.json` #12)
- 实现"发送消息"功能 (`feature_list.json` #15)
- 发现 CSS 主题系统需要重构, 已完成
- 测试通过: 新对话 + 发送消息 + 接收回复
- Git commit: "feat: basic chat interface with theme support"

Session 3 (2026-03-26 09:00)

- 启动时发现 Session 2 留下了一个未完成的功能
- 修复: 侧边栏对话列表排序逻辑
- 实现"删除对话"功能 (`feature_list.json` #18)
- Git commit: "fix: sidebar ordering, feat: delete conversation"

7. 沙箱环境 (Sandbox)

7.1 为什么需要沙箱?

没有沙箱的风险:

- X Agent 执行 `rm -rf /` → 删除系统文件
- X Agent 安装恶意包 → 安全漏洞
- X Agent 访问敏感文件 → 数据泄露
- X Agent 占满 CPU → 影响其他服务
- X Agent 的 bug 破坏宿主环境

有沙箱的好处:

- ✓ 安全隔离: agent 代码在容器中运行
- ✓ 可扩展: 并行创建多个沙箱环境
- ✓ 可复现: 相同的环境配置
- ✓ 可销毁: 任务完成后清理环境

7.2 沙箱的配置层次

沙箱配置层次

基础层：操作系统隔离

- └ Docker 容器 / 微虚拟机
- └ 网络隔离（可选）
- └ 资源限制（CPU/内存/磁盘配额）

工具层：预装开发工具

- └ 语言运行时（Python, Node.js, Go, Java...）
- └ 版本控制（git）
- └ 测试框架（pytest, jest...）
- └ 代码检查（linter, formatter）
- └ 浏览器（Chrome/Chromium for Puppeteer）

安全层：权限控制

- └ 命令白名单/黑名单
- └ 文件系统访问范围限制
- └ 网络出站规则
- └ 敏感操作审批

可观测层：审计与追踪

- └ 所有命令执行日志
- └ 文件修改记录
- └ 资源使用监控

代表实现：Daytona（TerminalBench 使用）、E2B、Modal、Fly.io。

第三篇：高级机制与实战方法论

8. 上下文腐烂 (Context Rot) 与对抗策略

8.1 什么是上下文腐烂?

Context Rot: 随着上下文窗口被填充, 模型的推理和任务完成能力逐渐退化。

原因:

1. 注意力稀释 → 早期重要指令被"挤到"注意力边缘
2. 噪音累积 → 大量工具输出占据 token 空间
3. 指令遗忘 → 系统约束在长上下文中被忽略
4. 自我矛盾 → 模型在长上下文中前后不一致

表现:

- 开始很聪明, 越工作越"笨"
- 重复已经做过的操作
- 忘记早期的约束和指令
- 开始产生幻觉

8.2 四大对抗策略

策略 1: 压缩 (Compaction)

当 token 使用量超过上下文窗口的 90% 时:

原始上下文 (50K tokens):

[系统指令] [对话 1-5] [工具输出 1-10] [对话 6-20] [工具输出 11-30]

↓ 压缩为

压缩后 (15K tokens):

[系统指令] [摘要: 对话 1-20 的关键信息] [最新 3 轮完整对话]

Harness 实现:

```
if token_count > 0.9 * max_tokens:
    summary = llm.summarize(early_conversations)
    context = [system_prompt, summary, recent_conversations]
```

策略 2: 工具输出截断 (Tool Call Offloading)

问题：一次 `bash` 命令可能返回 `10,000` 行输出，全部保留会快速耗尽上下文。

解决方案：



策略 3：技能渐进加载 (Progressive Skill Disclosure)

问题：启动时加载所有工具描述 = 消耗大量 `token` = 上下文还没开始工作就烂了。

解决方案：

启动时只加载：

- 基础工具 (`bash`, `read`, `write`, `edit`)
- 当前任务相关的技能描述

按需加载（当 `agent` 需要时）：

- 浏览器自动化技能
- 特定语言的构建工具
- MCP 服务器连接

→ 技能 (`Skill`) = Harness 级别的延迟加载原语

策略 4：Ralph Loop 重置

最激进的策略：不尝试修复腐烂的上下文，直接重开。

当上下文质量严重退化时：

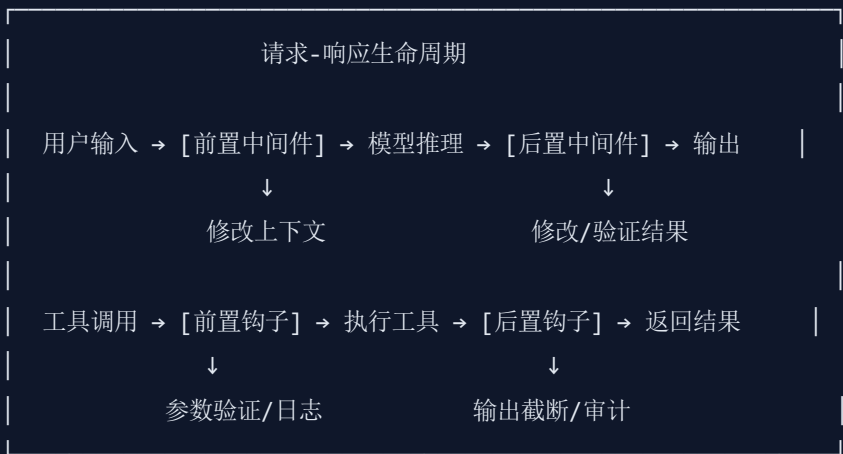
- 1. 将当前状态持久化到文件系统
- 2. 开启全新的上下文窗口
- 3. 只注入：原始 prompt + 进度文件 + git log
- 4. Agent 在干净的上下文中继续工作

→ 用"干净重启"替代"带病运行"

9. 中间件/钩子系统（Middleware & Hooks）

9.1 什么是 Harness 中间件？

中间件 = 在模型调用和工具调用前后执行的确定性逻辑



9.2 实战中间件清单

中间件	触发时机	功能	来源
PreCompletionChecklist	Agent 试图退出前	强制 agent 运行验证步骤	LangChain
LoopDetection	每次工具调用后	检测同一文件的重复编辑	LangChain
LocalContext	Agent 启动时	发现工作目录、可用工具	LangChain
TimeBudgetWarning	定时	注入时间预算提醒	LangChain
ToolCallOffloading	工具返回后	截断过长的工具输出	多个实现
Compaction	上下文快满时	压缩历史对话	Claude Code
Ralph Loop Hook	Agent 试图退出时	拦截退出，重开上下文	Anthropic

9.3 LoopDetection 中间件详解

问题: Agent 的"死亡螺旋" (Doom Loop)

- Agent 尝试修复一个 bug
- 修复尝试失败
- Agent 做微小调整再次尝试
- 再次失败
- 重复 10+ 次, 浪费大量 token 和时间

例子:

```
Edit file.py (attempt 1) → test fails
Edit file.py (attempt 2) → test fails
Edit file.py (attempt 3) → test fails
... (10 more times with minor variations)
```

LoopDetection 中间件:

追踪每个文件的编辑次数

当 `edit_count(file) > N` 时:

注入提醒: "你已经编辑了这个文件 {N} 次, 考虑换一种方法"

→ 引导 agent 退后一步, 重新思考策略

9.4 PreCompletionChecklist 中间件详解

问题: Agent 写完代码就直接说"我完成了", 没有验证。

解决方案 - 拦截 agent 的退出行为:

Agent: "任务完成, 修改已提交。"

↓ 中间件拦截

中间件注入: "在退出之前, 请确认以下清单:

- 你是否运行了测试?
- 测试是否全部通过?
- 你是否对照原始需求验证了输出?
- 是否有边界情况需要检查? "

↓

Agent: "让我运行测试...发现一个失败...修复...再次运行...全部通过。现在任务完成。"

10. NLAH — 自然语言 Agent Harness (2026 前沿)

10.1 问题：传统 Harness 不可移植

当前 Harness 的问题：

Claude Code 的 Harness → 用 Python 写的，绑定 Claude API
Codex 的 Harness → 用 TypeScript 写的，绑定 OpenAI API
OpenCode 的 Harness → 用 Go 写的，特定运行时

- 无法比较（两个系统表面差一个设计选择，实际差整个控制器）
- 无法移植（从一个运行时搬到另一个要重写）
- 无法研究（Harness 逻辑埋在代码里，无法系统消融实验）

10.2 NLAH 的五个核心组件

来自清华大学的论文 (arXiv:2603.25723, 2026)：

NLAH (自然语言 Agent Harness)

1. Contracts (契约)

- └ 必需的输入/输出
- └ 验证门控条件
- └ 权限边界
- └ 停止规则

2. Roles (角色)

- └ Solver: 解决问题
- └ Verifier: 验证结果
- └ Researcher: 研究背景
- └ Orchestrator: 编排协调
- 角色不重叠, 各有明确职责

3. Stage Structure (阶段结构)

- └ Plan → Execute → Verify → Repair
- └ 明确的工作流拓扑

4. Adapters & Scripts (适配器与脚本)

- └ 命名钩子: 测试、验证器、解析器
- └ 确定性操作的代码接口

5. State Semantics (状态语义)

- └ 跨步骤持久化什么
- └ 如何通过路径和清单重新打开状态

10.3 智能运行时 (IHR)



10.4 NLAH 的实验结果

实验	基准	结果
完整 IHR vs 无 Harness 逻辑	SWE-bench	16.3M tokens (有) vs 1.2M tokens (无) , 证明真正的多阶段编排
模块消融	SWE-bench	各模块可独立增减, 效果可量化
代码→文本迁移	OSWorld	30.4% → 47.2% (从原生代码迁移到 NLAH 表示后性能提升)

核心发现：自然语言表示的 Harness 可以在保持功能的同时**提升**可靠性，因为它将控制从截图修复循环转移到了文件支持的验证。

第四篇：工业实践与案例研究

11. Anthropic 的长期运行 Agent 模式

11.1 核心问题

"想象一个软件项目由轮班工程师负责，每位新工程师到岗时对前一个班次发生的事情毫无记忆。"

— Anthropic, "Effective Harnesses for Long-Running Agents" (2025)

长期运行 Agent 的三大失败模式：

失败 1：试图一次做完所有事（One-Shot 倾向）

- 上下文在中途耗尽 → 下一个 Agent 面对半成品且无文档
- 花大量时间猜测发生了什么

失败 2：过早宣布完成（Premature Victory）

- Agent 看到一些进展就认为任务完成
- 实际上大量功能尚未实现

失败 3：不做端到端测试

- Agent 修改代码后只做单元测试
- 不验证功能是否端到端工作
- 标记为完成的功能实际有 bug

11.2 双 Agent 解决方案

Anthropic 长期运行 Agent 架构

第一次会话：初始化 Agent (Initializer)

- └ 创建 `init.sh` (启动开发服务器的脚本)
- └ 创建 `claude-progress.txt` (进度记录文件)
- └ 创建 `feature_list.json` (功能清单, 全部标记为 `failing`)
 - └ 例: 200+ 个功能点, 每个都有精确的验证步骤
- └ 创建初始 `git commit`

后续每次会话：编码 Agent (Coder)

- └ 第一步：了解状态
 - └ `pwd` (确认工作目录)
 - └ 读取 `claude-progress.txt`
 - └ 读取 `feature_list.json`
 - └ `git log --oneline -20`
- └ 第二步：验证基础功能
 - └ 运行 `init.sh` 启动开发服务器
 - └ 用 `Puppeteer` 做基础端到端测试
- └ 第三步：选择一个功能
 - └ 从 `feature_list.json` 中选择最高优先级的 `failing` 功能
- └ 第四步：实现功能
 - └ 编写代码
 - └ 运行测试
 - └ 端到端验证 (用浏览器自动化)
- └ 第五步：记录和提交
 - └ 更新 `feature_list.json` (将完成的功能标记为 `passing`)
 - └ 更新 `claude-progress.txt`
 - └ `git commit` (描述性提交信息)

11.3 功能清单的设计智慧

```
{
  "category": "functional",
  "description": "New chat button creates a fresh conversation",
  "steps": [
    "Navigate to main interface",
    "Click the 'New Chat' button",
    "Verify a new conversation is created",
    "Check that chat area shows welcome state",
    "Verify conversation appears in sidebar"
  ],
  "passes": false
}
```

为什么用 JSON 而不是 Markdown?

"After some experimentation, we landed on using JSON for this, as the model is less likely to inappropriately change or overwrite JSON files compared to Markdown files."

— Anthropic

关键约束：

- Agent 只能修改 `passes` 字段（不能删除或修改测试步骤）
- 使用强措辞："It is unacceptable to remove or edit tests because this could lead to missing or buggy functionality"

12. LangChain 的 Harness 优化实战

12.1 从 Top 30 到 Top 5

LangChain 在 TerminalBench 2.0 上的优化历程：

初始分数：52.8% (Top 30 开外)

模型：GPT-5.2-Codex (固定不变)

只优化 Harness

最终分数：66.5% (Top 5)

提升：+13.7 分

12.2 优化方法论：追踪分析循环

Trace Analyzer Skill（追踪分析技能）

Step 1: 收集追踪数据

└ 从 LangSmith 获取实验追踪

└ 包括：延迟、token 消耗、工具调用序列、错误

Step 2: 自动错误分析

└ 并行 spawn 多个分析 Agent

└ 每个 Agent 分析不同类型的失败

└ 主 Agent 综合发现 + 建议

└ 类似 boosting 算法：专注于之前犯的错误

Step 3: 定向改进

└ 人类可以参与验证（非必需但有帮助）

└ 避免过拟合到特定任务（损害泛化）

└ 自动追踪分析节省数小时手动调试时间

Step 4: 迭代

└ 重复 Step 1-3，每次聚焦不同的失败模式

12.3 具体改进措施

改进 1：构建-自验证循环

最常见的失败模式：

Agent 写了解决方案 → 读自己的代码 → "看起来不错" → 停止

改进后的系统 prompt：

"你的问题解决流程必须是：

1. 规划与发现：阅读任务、扫描代码库、制定计划

2. 构建：实现计划，同时构建测试

3. 验证：运行测试，对照原始需求（而非自己的代码）比较

4. 修复：分析错误，回到原始规格，修复问题"

改进 2：环境上下文注入

LocalContextMiddleware:

Agent 启动时自动:

- └ 扫描工作目录结构
- └ 发现可用的 Python/Node.js 版本
- └ 识别可用的测试框架
- └ 将这些信息注入上下文

→ 减少环境发现阶段的错误

改进 3: 时间预算管理

Agent 的天然缺陷: 不善于估计时间

时间预算中间件:

当剩余时间 < 30% 时:

注入提醒: "剩余时间不足, 请转向验证阶段"

当剩余时间 < 10% 时:

强制提醒: "请立即提交当前工作"

改进 4: 推理预算优化

GPT-5.2-Codex 有 4 个推理模式: low, medium, high, xhigh

实验发现:

- 全程 xhigh → 53.9% (因超时而失败)
- 全程 high → 63.6% (更好的时间利用)
- "推理三明治" xhigh-high-xhigh → 66.5%
 - └ 规划阶段用 xhigh (充分理解问题)
 - └ 执行阶段用 high (节省时间)
 - └ 验证阶段用 xhigh (仔细检查)

→ 关键洞察: 在正确的阶段花费正确的推理预算

13. OpenAI 的百万行代码 Harness

13.1 核心方法

OpenAI 团队 (2026) 用一个约束条件打造了 Harness: **"完全不手写代码"**。

5 个月后的结果: 维护一个 100 万行代码的真实产品。

他们的 Harness 三大组件：

1. 上下文工程（Context Engineering）

- └ 代码库中持续增强的知识库
- └ Agent 可访问的动态上下文
 - └ （可观测性数据、浏览器导航等）
- └ 让 Agent 始终拥有完成任务所需的背景信息

2. 架构约束（Architectural Constraints）

- └ LLM Agent 监控架构规则
- └ 确定性自定义 linter（代码风格/结构）
- └ 结构化测试（模块边界/数据结构稳定性）
- └ 双重保障：AI + 确定性检查

3. 垃圾回收（Garbage Collection）

- └ 定期运行的 Agent 查找不一致
- └ 发现文档过期 → 自动更新
- └ 发现架构约束违反 → 自动修复
- └ 对抗代码库的熵增和衰减

13.2 迭代式 Harness 改进

"When the agent struggles, we treat it as a signal: identify what is missing — tools, guardrails, documentation — and feed it back into the repository, always by having Codex itself write the fix."

— OpenAI

Harness 改进循环：

Agent 在任务中挣扎

↓

诊断缺失了什么（工具？护栏？文档？）

↓

让 Agent 自己写修复方案

↓

修复方案成为 Harness 的一部分

↓

下次 Agent 不会再犯同样的错误

= Mitchell Hashimoto 的原则："让 agent 永远不再犯那个特定的错误"

14. 核心设计原则汇总

原则	说明	来源
模型是半成品	Agent = Model + Harness，模型提供智能但不是完整产品	LangChain
文件系统为王	不需要向量数据库，文件系统是最基础的 Harness 原语	多个团队
增量优于一次	每次只做一个功能/一步操作，避免 one-shot	Anthropic
验证不可省略	每个操作后验证，用确定性工具而非仅靠模型判断	LangChain
错误即信号	Agent 犯错 = Harness 有缺陷，修复 Harness	Hashimoto
Harness 外部化	关键控制逻辑不应依赖模型"自觉遵守"，要在 Harness 层强制执行	NLAH
追踪驱动优化	用 Trace 分析失败模式，定向改进 Harness	LangChain
确定性 + AI 双保险	关键约束同时用代码检查和 AI 判断	OpenAI
为模型量身定制	不同模型需要不同的 Harness 配置（prompt 风格、工具描述、推理预算）	LangChain
护栏会过时	当前需要的中间件和护栏，随模型提升可能不再需要	LangChain

第五篇：研究前沿与开放问题

15. Harness 的进化方向

15.1 模型与 Harness 的协同进化

当前趋势：模型训练中考虑 Harness

Claude Code ↔ Claude 模型训练
Codex ↔ GPT 模型训练

反馈循环：

- 1. Harness 中发现有用的新原语
- 2. 下代模型训练中包含这些原语
- 3. 模型变得更擅长使用 Harness
- 4. Harness 发现更高层的原语
- 5. 循环继续...

副作用：

- 模型对特定 Harness 过拟合
- 换 Harness 可能性能下降
- 例：Codex 的 apply_patch 工具逻辑
(理论上智能模型应能适应不同 patch 方法，但训练中绑定了特定方法)

15.2 开放研究问题

问题	说明
并行 Agent 协作	100+ Agent 同时在共享代码库上工作如何协调？
自动 Harness 优化	Agent 自动分析自己的 Trace 并修复 Harness 级别缺陷
动态工具组装	根据任务按需组装正确的工具和上下文，而非预配置
跨模型 Harness 通用性	一个 Harness 如何在不同模型间保持高效？
Harness 标准化	NLAH 之后，是否有统一的 Harness 描述标准？
长期记忆的可扩展性	文件系统记忆在超大规模（百万文件）下如何管理？
Harness 可靠性验证	如何系统化测试 Harness 本身的正确性？

15.3 未来展望

短期（2026）：

- Harness 工程成为 AI 工程师的核心技能
- 企业级 Harness 模板出现（类似今天的 Service Template）
- NLAH 标准化推进

中期（2027-2028）：

- 部分 Harness 功能被模型吸收（如自我验证、规划）
- 但系统级设计（沙箱、编排、多界面）仍是 Harness 领域
- Harness 即产品（Claude Code、Codex 本身就是 Harness 产品）

长期（2029+）：

- Harness 与传统软件工程融合
- "Harness Engineer" 成为标准角色
- 像今天的前端框架一样，出现 Harness 框架生态

16. 实践检查清单

16.1 构建你的第一个 Harness

第一步：最小可用 Harness

- ReAct 循环（while loop + 模型调用 + 工具执行）
- 基础工具：bash + read_file + write_file + edit_file
- 简单的系统 prompt
- 基本的错误处理

第二步：添加持久化

- 文件系统访问
- 进度记录文件
- Git 集成
- AGENTS.md 支持

第三步：添加可靠性

- 上下文压缩
- 工具输出截断
- 自我验证提示
- Pre-completion checklist

第四步：添加高级功能

- 沙箱环境
- Ralph Loop（跨上下文窗口延续）
- 环境发现中间件
- 死亡螺旋检测

第五步：添加可观测性

- Trace 收集
- 自动错误分析
- Harness 改进循环
- 性能基准测试

16.2 Harness 评估维度

评估你的 Harness 是否优秀：

可靠性 (Reliability)

- └ Agent 在各种输入下一致地产生正确结果
- └ 跨会话工作不丢失进度
- └ 错误能被检测和恢复

效率 (Efficiency)

- └ Token 使用合理（不浪费在噪音上）
- └ 时间管理良好（在预算内完成）
- └ 推理预算分配得当

安全性 (Safety)

- └ 在沙箱中执行
- └ 关键操作有确认
- └ 不会执行破坏性操作

可维护性 (Maintainability)

- └ Harness 配置可读、可修改
- └ 可以独立于模型调整 Harness
- └ 有追踪和诊断能力

可移植性 (Portability)

- └ 可以换模型而不重写 Harness
- └ 可以在不同环境运行
- └ 配置与代码分离

17. 关键资源索引

资源	类型	核心贡献
NLAH (arXiv:2603.25723)	论文	自然语言 Harness 理论框架 + IHR 运行时
Anthropic: Effective Harnesses for Long-Running Agents	博客	双 Agent 模式 + 进度文件 + 功能清单
LangChain: Improving Deep Agents with Harness Engineering	博客	Trace 分析 + 中间件设计 + 推理预算优化
LangChain: Anatomy of an Agent Harness	博客	Harness 完整解剖 + 从模型需求反推设计
Decoding AI: Agentic Harness Engineering	文章	LLM as OS + 文件系统记忆 + 完整架构图
OpenAI: Harness Engineering with Codex	博客	百万行代码 + 确定性+AI 双保险
Martin Fowler: Harness Engineering	文章	Harness 作为新的 Service Template
Mitchell Hashimoto: My AI Adoption Journey	博客	Harness Engineering 定义 + 六步实践法
Anthropic: Building Effective AI Agents	博客	Agent 设计模式 + 渐进式架构
Anthropic: Effective Context Engineering	博客	上下文工程实践 (Harness 的核心子集)

整理自 arXiv 论文、工程博客、工业实践报告，2026-04-02 全文约 41,000 字